

Hardware und Software

Für den Praktikumsaufbau werden folgende Hard- und Softwarekomponenten benötigt.

Hardware

- STM32LR476RG Nucleo Board
- 8 rote LEDs
- 8 grüne LEDs
- 8 Kippschalter
- 1 Analog Potentiometer
- 1 Digital Potentiometer (Drehgeber)

Software

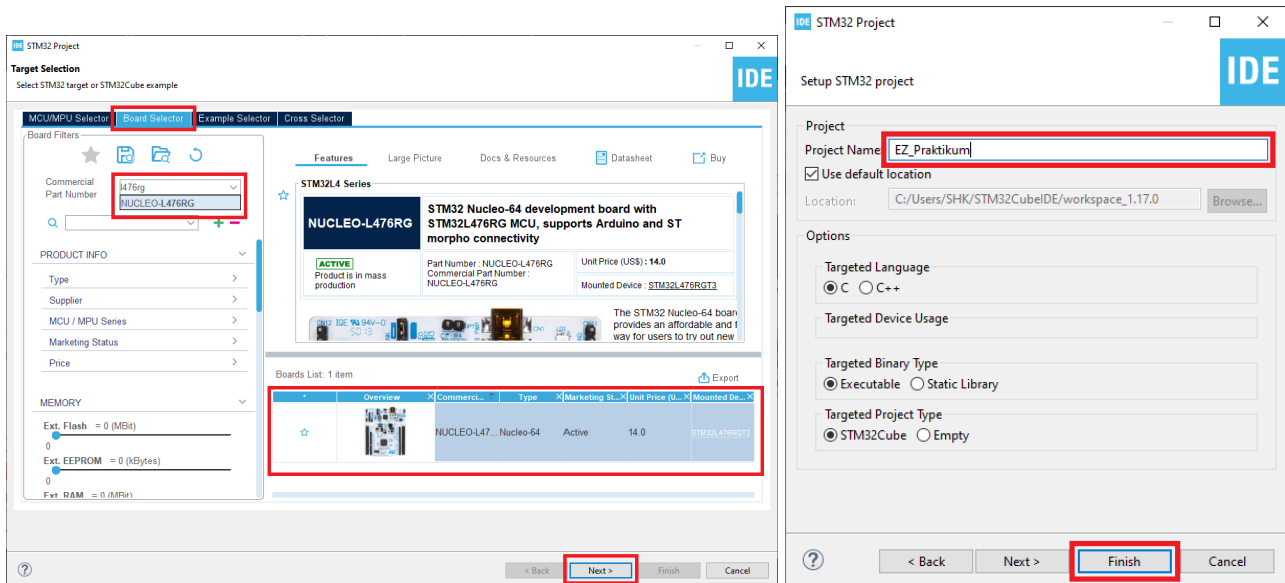
STM32CubeIDE verfügbar für Windows, Linux und macOS
<https://www.st.com/en/development-tools/stm32cubeide.html>

Anlegen eines STM32 Projekts

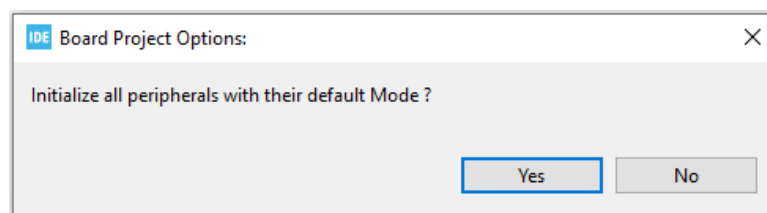
Um ein neues STM32CubeIDE Projekt anzulegen, starten Sie zunächst die IDE.

Anschließend wählen Sie **File** → **New** → **STM32 Project** und im neu geöffneten Fenster unter **Target Selection** → **Board Selector** das Board **Nucleo-L476RG** aus.

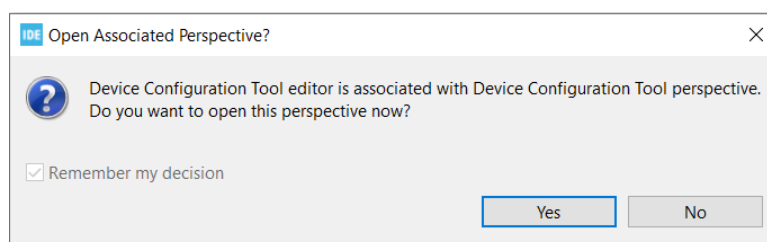
Über die Suchfunktion kann das entsprechende Board schnell gefunden werden. Anschließend wählen Sie einen Projektnamen und bestätigen mit **Finish**.



Die IDE fragt, ob die Peripherie mit den Default-Einstellungen initialisiert werden soll. Bestätigen Sie dies.



Die IDE fragt, ob in die CubeMX-Perspektive gewechselt werden soll. Bestätigen Sie auch dies.



Ein neues STM32 Projekt wurde angelegt und es kann mit den Einstellungen für die Pinbelegung fortgefahren werden.

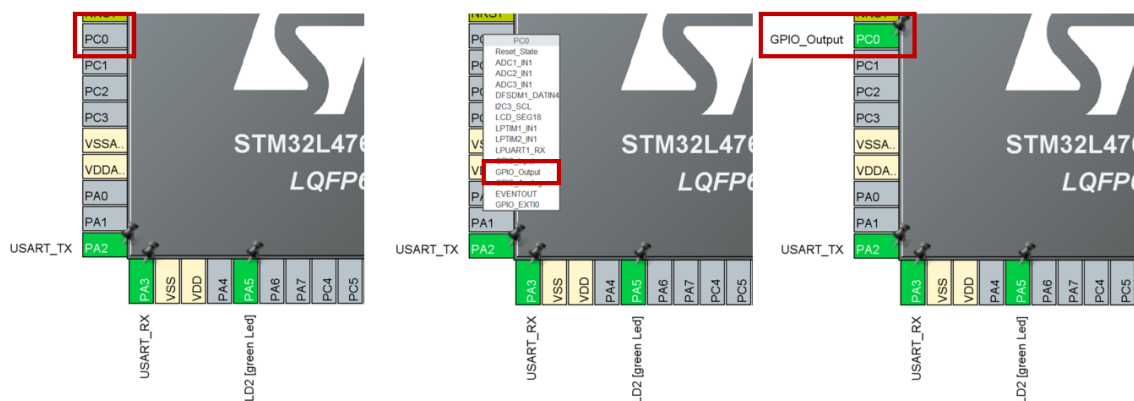
Pinbelegung

Die Pinbelegung an den Praktikumsaufbauten ist wie folgt. Die LEDs sind an GPIOC Pin 0 bis 15 angeschlossen, die Schalter an GPIOB Pin 8 bis 15. Das analoge Potentiometer ist an GPIOB, Pin 0 angeschlossen. Das digitale Potentiometer (Drehgeber) ist an GPIOA Pin 0 und 1 angeschlossen. GPIOA Pin 2 und 3 dienen der UART Kommunikation. GPIOA Pin 6,7 und GPIOB Pin 4,5 werden für eine Linear-Achse mit Encoder und einen externen Drehgeber konfiguriert. GPIOB Pin 7 und GPIOA Pin 15 werden für eine PWM-Steuerung verwendet. Auf GPIOA Pin 11 und 12 steht ein CAN-Bus zur Verfügung.

GPIO A	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Tac			CAN					Enc				UART		D	
GPIO B	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	Schalter								PWM		End					A
GPIO C	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	LEDs (grün)								LEDs (rot)							

Anpassen der Pinbelegung im CubeMX-Tool

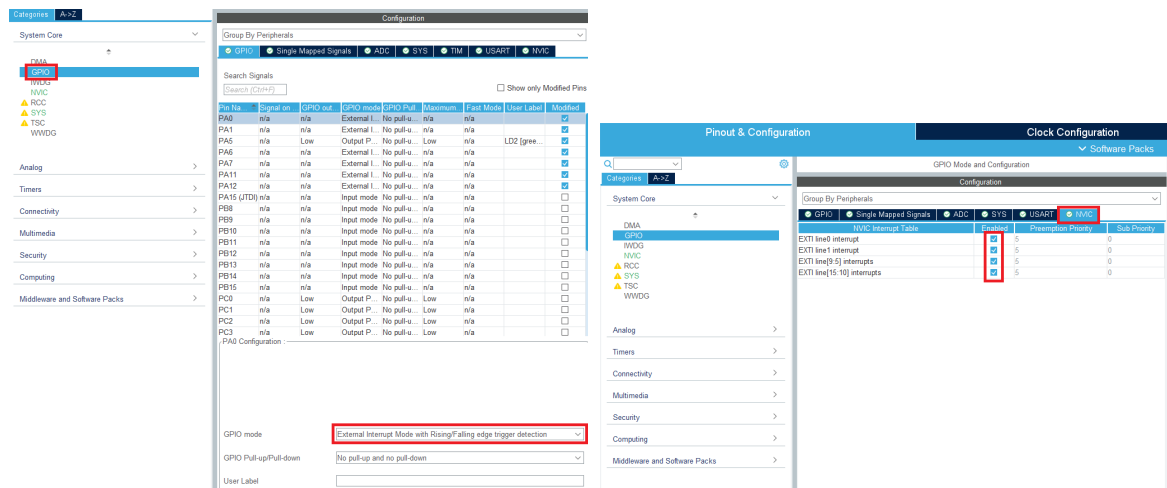
Öffnen Sie die CubeMX Perspektive durch einen Doppelklick auf die `.ioc-Datei` im Project Explorer auf der linken Seite. Durch einen Klick auf den jeweiligen Pin werden die Belegungsoptionen angezeigt.



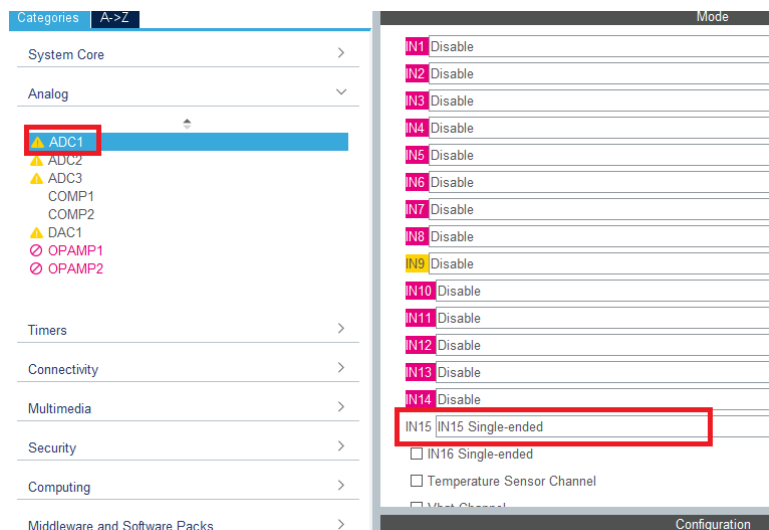
Die Pins **PC0 bis PC15** müssen als **GPIO_Output** konfiguriert werden. Dazu wird auf einen entsprechenden Pin geklickt und die Option GPIO Output wird ausgewählt. Durch ein erneutes Klicken auf die ausgewählte Option kann die Auswahl auch wieder rückgängig gemacht werden.

Die Pins **PA0, PA1, PA6, PA7, PA11, PA12, PA15, PB4, PB5, PB8, PB9** und **PB10** müssen als **GPIO_EXTI** konfiguriert werden und **PB11 bis PB15** als **GPIO_Input**.

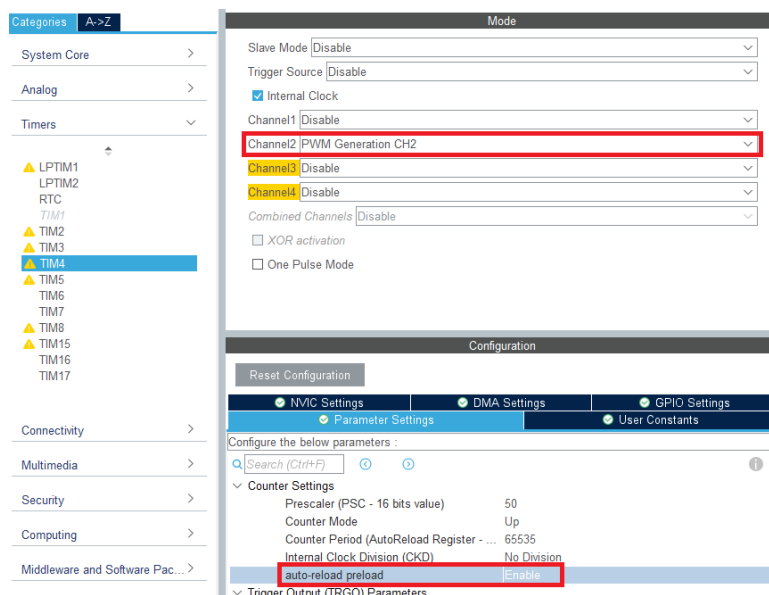
Um die **GPIO_EXTI** Interrupts später verwenden zu können muss für jeden dieser Pins der **GPIO Mode** auf *External Interrupt Mode with Rising /Falling edge trigger detection* geändert werden und unter dem Reiter **NVIC** ein Haken für alle konfigurierten Pins gesetzt werden.



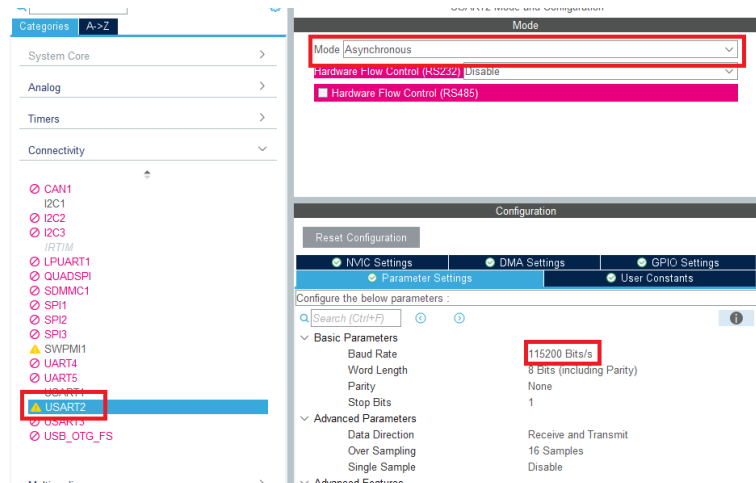
Pin PB0 muss als **ADC1 IN15** konfiguriert werden. Auf der linken Seite unter **Analog** → **ADC1** muss dabei für den Eingang IN15 die Einstellung **IN15 Single-ended** ausgewählt werden.



Pin PB7 wird als **TIM4_CH2** konfiguriert und unter **Timers** → **TIM4** muss **Channel2** auf **PWM Generation CH2** gesetzt werden und die Option **auto-reload preload** aktiviert werden.

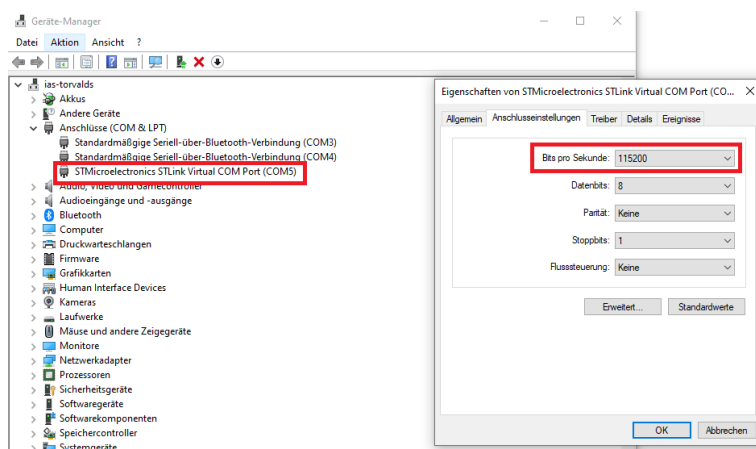


Die Pins **PA2** und **PA3** werden automatisch für die UART-Kommunikation als **USART_TX** und **USART_RX** konfiguriert, wenn die Default-Initialisierung der Peripherie bei der Erstellung des Projekts bestätigt wurde. Auf der linken Seite unter **Connectivity** → **USART2** können weitere Einstellungen vorgenommen werden. Wenn der Modus für **USART2** noch nicht gesetzt ist (auf Disable steht) muss er auf **Asynchronous** gestellt werden. Die **Baud-Rate** sollte auf **115200 Bits/s** stehen.



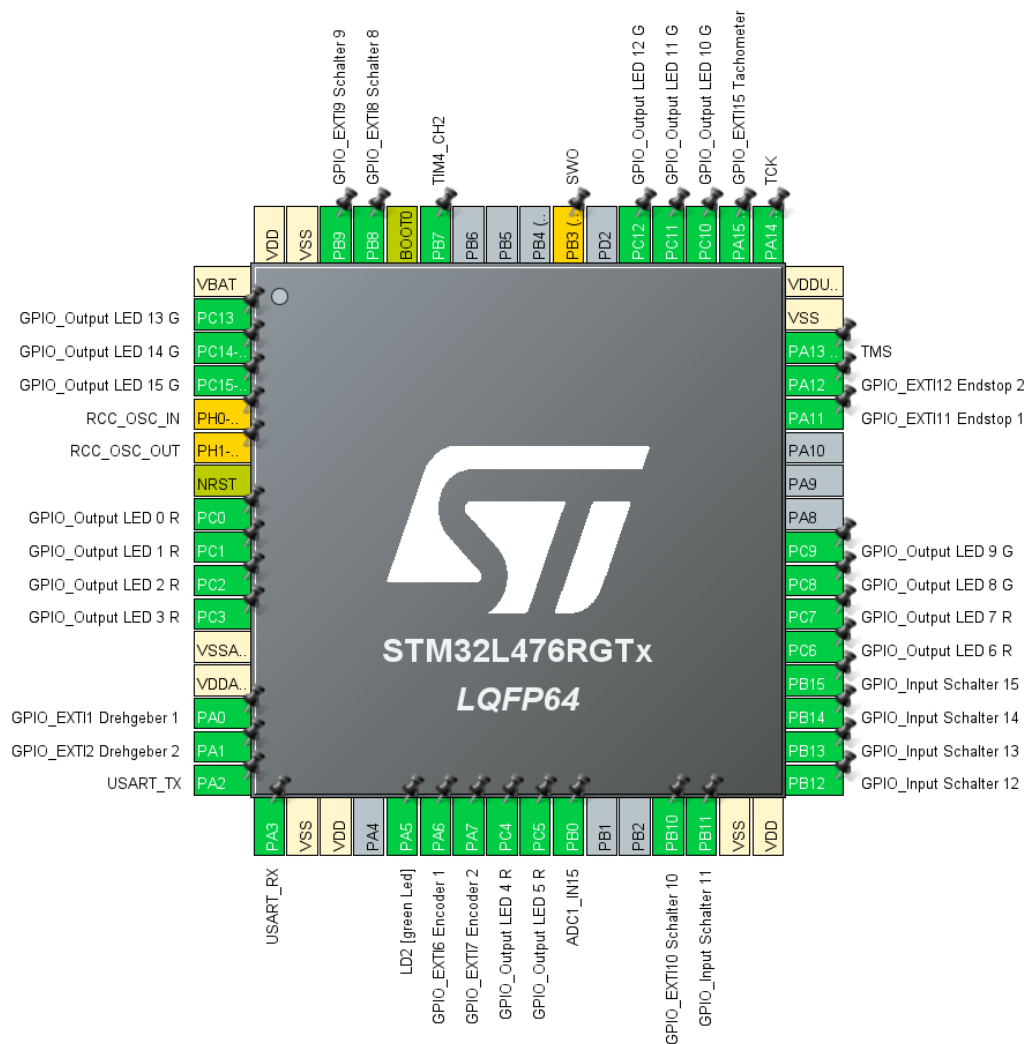
Die Baud-Rate muss im System korrekt für den Mikrocontroller eingestellt werden.

- Windows
 - a) Rechtsklick auf **Start**
 - b) **Geräte-Manager** öffnen
 - c) Unter **Anschlüsse (COM & LPT)** den Mikrocontroller (**STMicroelectronics STLink Virtual COM Port (COMx)**) wählen.
 - d) Auf den Reiter **Anschlusseinstellungen** wechseln.
 - e) Bits pro Sekunde auf **115200 Bit/s** einstellen



- Linux
 - a) Mit `ls /dev` die Liste der angeschlossenen Geräte anzeigen
 - b) Das richtige `tty` Terminal identifizieren. (entweder `ttyACMx` oder `ttyUSBx`)
 - c) Mit `stty -F /dev/ttyACMx` oder `stty -F /dev/ttyUSBx` die aktuelle Baud-Rate lesen
 - d) Wenn die Baud-Rate noch nicht identisch zu der in CubeMX eingestellten ist, kann sie mit `stty -F /dev/ttyACMx 115200` oder `stty -F /dev/ttyUSBx 115200` gesetzt werden.
- MacOS (nicht getestet)
 - a) Mit `ls /dev` die Liste der angeschlossenen Geräte anzeigen
 - b) Das richtige `tty` Terminal identifizieren. (`tty.usbserial-xxxxxx`)
 - c) Mit `stty -F /dev/tty.usbserial-xxxxxx` die aktuelle Baud-Rate lesen
 - d) Wenn die Baud-Rate noch nicht identisch zu der in CubeMX eingestellten ist, kann sie mit `stty -F /dev/tty.usbserial-xxxxxx 115200` gesetzt werden.

Die folgende Abbildung zeigt abschließend die gesamte fertige Pinbelegung in der CubeMX-Ansicht.



Pin	Funktion	Pin	Funktion	Pin	Funktion	Pin	Funktion
PA0	EXTI_0	PB0	ADC1_IN15	PC0	GPIO_OUT	PD2	-
PA1	EXTI_1	PB1	-	PC1	GPIO_OUT	PH0	RCC_OSC_IN
PA2	USART_TX	PB2	-	PC2	GPIO_OUT	PH1	RCC_OSC_OUT
PA3	USART_RX	PB3	SWO	PC3	GPIO_OUT		
PA4	-	PB4	EXTI_4	PC4	GPIO_OUT		
PA5	LD2	PB5	EXTI_5	PC5	GPIO_OUT		
PA6	EXTI_6	PB6	-	PC6	GPIO_OUT		
PA7	EXTI_7	PB7	TIM4_CH2	PC7	GPIO_OUT		
PA8	-	PB8	EXTI_8	PC8	GPIO_OUT		
PA9	-	PB9	EXTI_9	PC9	GPIO_OUT		
PA10	-	PB10	EXTI_10	PC10	GPIO_OUT		
PA11	EXTI_11	PB11	GPIO_IN	PC11	GPIO_OUT		
PA12	EXTI_12	PB12	GPIO_IN	PC12	GPIO_OUT		
PA13	TMS	PB13	GPIO_IN	PC13	GPIO_OUT		
PA14	TCK	PB14	GPIO_IN	PC14	GPIO_OUT		
PA15	EXTI_15	PB15	GPIO_IN	PC15	GPIO_OUT		

freeRTOS

Um freeRTOS nutzen zu können muss in der Kategorie **Middleware and Software Packs**, **FREERTOS** ausgewählt werden und von *Disable* auf *CMSIS_V2* geschaltet werden. Um die von freeRTOS generierten Tasks steuern zu können, müssen unter Include Parameters folgende Optionen auf **Enabled** stehen:

- vTaskPrioritySet
- vTaskDelete
- vTaskSuspend
- vTaskDelay
- vTaskDelayUntil
- xEventGroupSetBitFromISR
- xQueueGetMutexHolder
- xSemaphoreGetMutexHolder

FREERTOS Mode and Configuration

Mode

Interface: CMSIS_V2

Configuration

Reset Configuration

☒ Mutexes	☒ Events	☒ FreeRTOS Heap Usage
☒ User Constants	☒ Tasks and Queues	☒ Timers and Semaphores
☒ Config parameters	☒ Include parameters	☒ Advanced settings

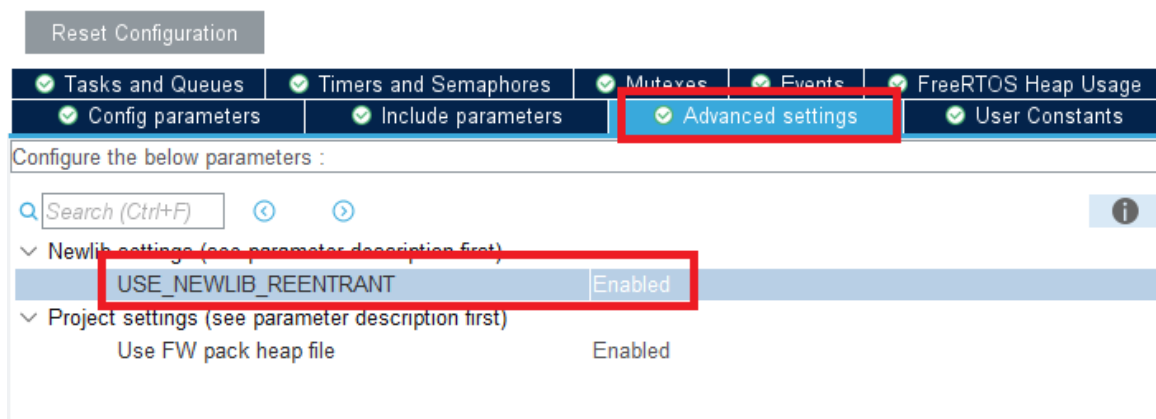
Configure the below parameters :

Search (Ctrl+F)

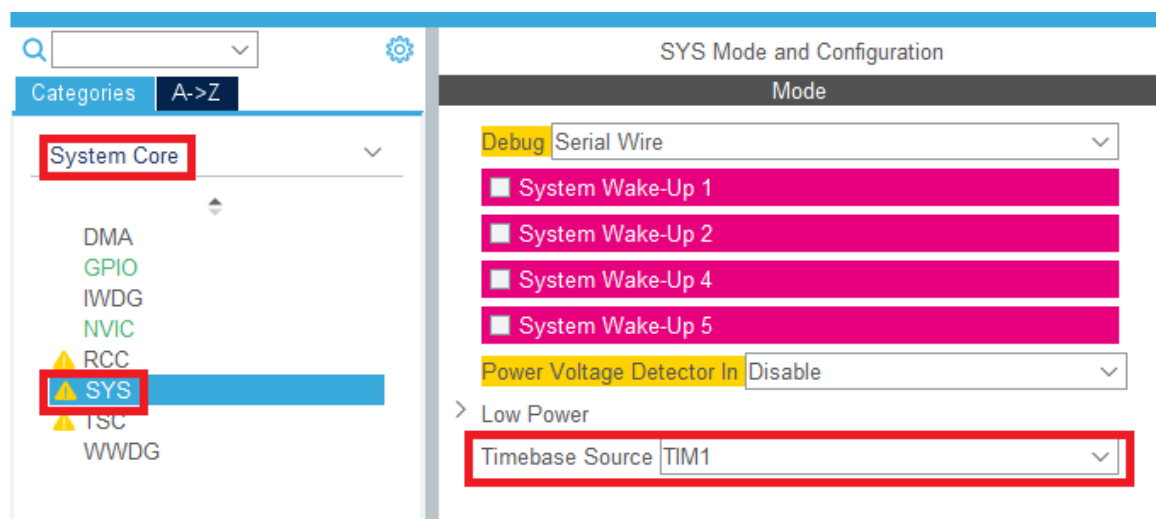
Include definitions

vTaskPrioritySet	Enabled
uxTaskPriorityGet	Enabled
vTaskDelete	Enabled
vTaskCleanUpResources	Disabled
vTaskSuspend	Enabled
vTaskDelayUntil	Enabled
vTaskDelay	Enabled
xTaskGetSchedulerState	Enabled
xTaskResumeFromISR	Enabled
xQueueGetMutexHolder	Enabled
xSemaphoreGetMutexHolder	Enabled
pcTaskGetTaskName	Disabled
uxTaskGetStackHighWaterMark	Enabled
xTaskGetCurrentTaskHandle	Enabled
eTaskGetState	Enabled
xEventGroupSetBitFromISR	Enabled
xTimerPendFunctionCall	Enabled
xTaskAbortDelay	Disabled
xTaskGetHandle	Disabled
uxTaskGetStackHighWaterMark2	Disabled

Anschließend wird unter dem Reiter **Advanced settings** die Option **USE_NEWLIB_REENTRANT** aktiviert um spätere Fehlermeldungen zu vermeiden.



Zum Schluss muss die Systemuhr für Echtzeitznutzung angepasst werden. Dazu muss in der Kategorie **System Core** unter **SYS** die **Timebase Source** von *SysTick* auf *TIM1* gewechselt werden.



Code generieren

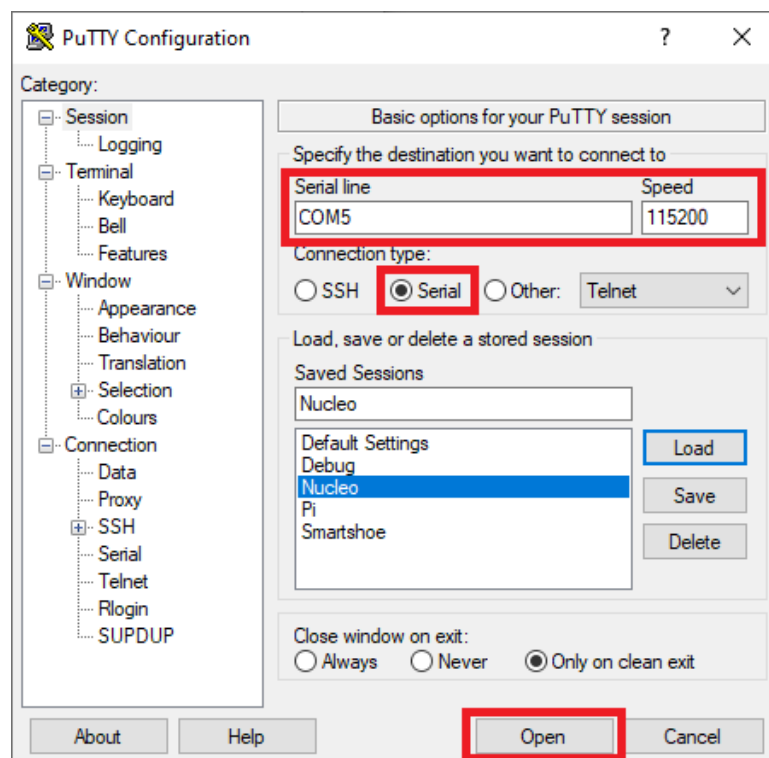
Jetzt sollte das Projekt gespeichert werden. Dadurch wird automatisch der entsprechende C-Code erzeugt. Die Einstellung der des Mikrocontrollers ist somit abgeschlossen. Sollte die Codegenerierung nicht automatisch erfolgen kann sie über die Menüleiste **Project** → **Generate Code** manuell getriggert werden.

Konsole für die Ausgabe

Da die integrierte Konsole der IDE Probleme verursachen kann, wird empfohlen ein externes Terminal zu verwenden (z.B. Putty). Wenn ein externes Terminal geöffnet wird, müssen die, während der Konfiguration eingestellten, Werte für das serielle Interface und die Baud-Rate eingestellt werden. Interface:

- Windows COMx
- Linux /dev/ttyACMx oder /dev/ttyUSBx
- MacOS /dev/tty.usbserial-xxxxxx

Baud-Rate: 115200



C printf

Um die Funktion printf wie gewohnt nutzen zu können, müssen ein paar Änderungen vorgenommen werden, wie unter <https://shawnhymel.com/1873/how-to-use-printf-on-stm32/> beschrieben.

Im Ordner Inc muss eine Datei retarget.h hinzugefügt werden.

retarget.h:

```
// All credit to Carmine Novello for this code .  
// https://github.com/cnovello/mastering-stm32/blob/  
// master/nucleo-f030R8/system/include/retarget/retarget.h  
  
#ifndef _RETARGET_H_  
#define _RETARGET_H_  
  
#include "stm32l4xx_hal.h"  
#include <sys/stat.h>  
  
void RetargetInit(UART_HandleTypeDef *huart);  
int _isatty(int fd);  
int _write(int fd, char* ptr, int len);  
int _close(int fd);  
int _lseek( int fd, int ptr, int dir);  
int _read(int fd, char* ptr, int len);  
int _fstat(int fd, struct stat* st);  
  
# endif //ifndef _RETARGET_H_
```

Im Ordner Src muss eine Datei `retarget.c` hinzugefügt werden.
retarget.c:

```
#include <_ansi.h>
#include <_syslist.h>
#include <errno.h>
#include <sys/time.h>
#include <sys/times.h>
#include <limits.h>
#include <signal.h>
#include <retarget.h>
#include <stdint.h>
#include <stdio.h>

#if !defined(OS_USE_SEMIHOSTING)
#define STDIN_FILENO 0
#define STDOUT_FILENO 1
#define STDERR_FILENO 2

UART_HandleTypeDef *gHuart;

void RetargetInit(UART_HandleTypeDef *huart){
    gHuart = huart;

    /* Disable I/O buffering for STDOUT stream, so that,
     * chars are sent out as soon as they are printed. */
    setvbuf(stdout, NULL, _IONBF, 0);
}

int _isatty(int fd){
    if ( fd >= STDIN_FILENO && fd <= STDERR_FILENO )
        return 1;

    errno = EBADF;
    return 0;
}

int _write(int fd, char* ptr, int len){
    HAL_StatusTypeDef hstatus;
    if(fd == STDOUT_FILENO || fd == STDERR_FILENO){
        hstatus = HAL_UART_Transmit(gHuart, (uint8_t *) ptr, len, HAL_MAX_DELAY);
        if(hstatus == HAL_OK)
            return len;
        else
            return EIO;
    }
    errno = EBADF;
    return -1;
}

int _close(int fd){
    if(fd >= STDIN_FILENO && fd <= STDERR_FILENO)
        return 0;

    errno = EBADF;
    return -1;
}
```

```
int _lseek(int fd, int ptr, int dir){
    (void) fd;
    (void) ptr;
    (void) dir;

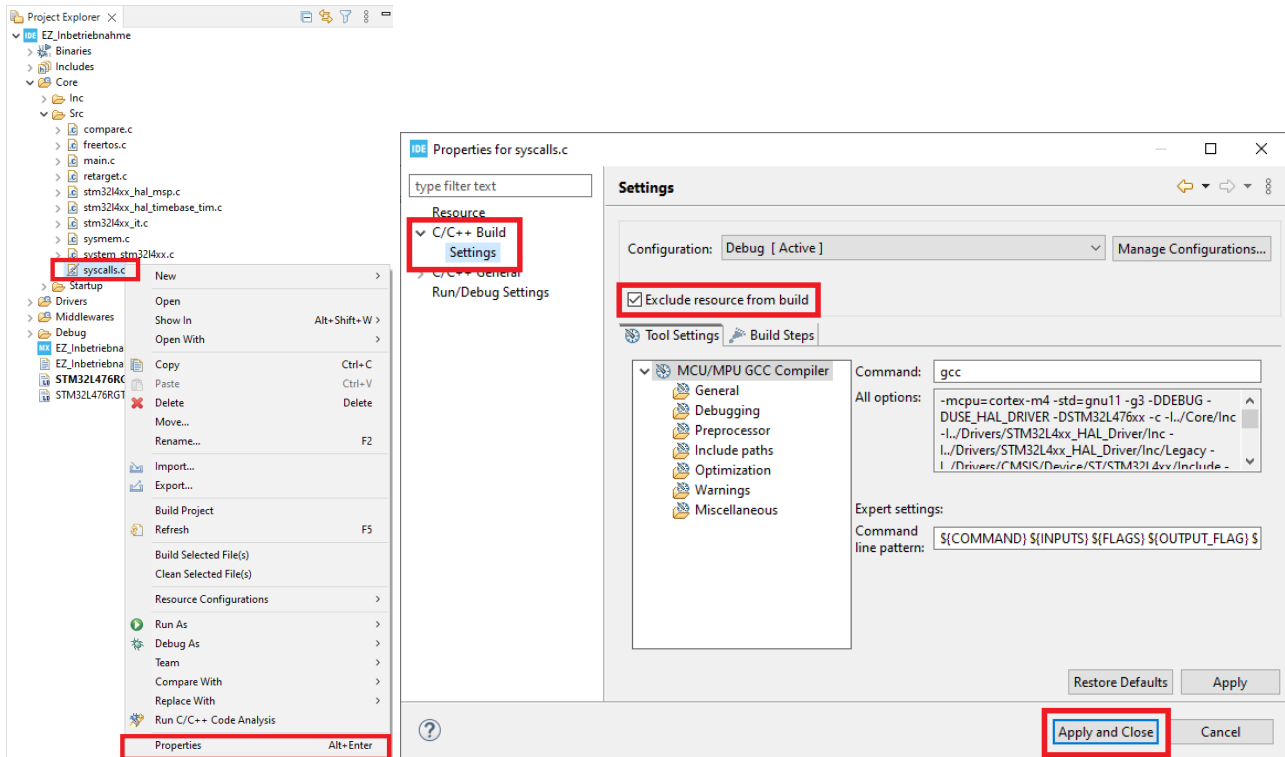
    errno = EBADF;
    return -1;
}

int _read(int fd, char* ptr, int len){
    HAL_StatusTypeDef hstatus;
    if(fd == STDIN_FILENO){
        hstatus = HAL_UART_Receive (gHuart, (uint8_t *) ptr, 1, HAL_MAX_DELAY);
        if(hstatus == HAL_OK)
            return 1;
        else
            return EIO;
    }
    errno = EBADF;
    return -1;
}

int _fstat(int fd, struct stat* st){
    if (fd >= STDIN_FILENO && fd <= STDERR_FILENO){
        st->st_mode = S_IFCHR;
        return 0;
    }
    errno = EBADF;
    return 0;
}

#endif // #if !defined(OS_USE_SEMIHOSTING)
```

Die Datei `syscalls.c` muss über einen Rechtsklick auf den Datei-Namen unter **Properties** → **C/C++ Build** → **Settings** → **Exclude Resource from Build** deaktiviert werden. Sie wird dann ausgegraut angezeigt.



In der `main.c`-Datei muss unter Private Includes die `retarget.h`-Datei eingebunden werden.

```
/* Private includes - - - - -  
- - */  
/* USER CODE BEGIN Includes */  
#include <stdio.h>  
#include "retarget.h"  
/* USER CODE END Includes */
```

Außerdem muss die Initialisierungsfunktion `RetargetInit()` mit passendem Parameter aufgerufen werden.

```
/* USER CODE BEGIN 2 */  
RetargetInit(&huart2);  
/* USER CODE END 2 */
```

Die Funktionen `printf` und `scanf` können dann wie gewohnt verwendet werden.

Programmieren mit freeRTOS

In freeRTOS werden ihre Programme nicht mehr direkt aus einer `main()` Funktion gestartet, sondern in Tasks ausgeführt. Die STM32CubeIDE erzeugt standardmäßig eine *StartDefaultTask* mit der CMSIS Bibliothek, da wir direkt mit dem freeRTOS Befehlssatz arbeiten kann diese gelöscht/ignoriert werden. Um eine Task zu erstellen verwenden sie den Befehl `xTaskCreate()`. Dieser muss in der **main.c** zwischen *osKernelInitialize()* und *osKernelStart* aufgerufen werden. Die Befehlsreferenz für `xTaskCreate()` und alle weiteren freeRTOS Befehle finden sie unter:

freeRTOS Referenzen

Um auf die Hardware des STM32 zuzugreifen können sie wie in PE entweder die Register direkt nutzen, oder die HAL(Hardware Abstraction Layer)-Bibliothek benutzen. Die Dokumentation zum HAL finden sie unter:

HAL Referenzen

Ein Beispiel, wie ihre **main.c** aussehen sollte:

```
osKernelInitialize();

/* USER CODE BEGIN RTOS_MUTEX */
/* add mutexes, ... */
/* USER CODE END RTOS_MUTEX */

/* USER CODE BEGIN RTOS_SEMAPHORES */
/* add semaphores, ... */
/* USER CODE END RTOS_SEMAPHORES */

/* USER CODE BEGIN RTOS_TIMERS */
/* start timers, add new ones, ... */
/* USER CODE END RTOS_TIMERS */

/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
/* USER CODE END RTOS_QUEUES */

/* Create the thread(s) */
/* creation of defaultTask */
defaultTaskHandle = osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);

/* USER CODE BEGIN RTOS_THREADS */
/* add threads, ... */
printf("\e[1;1H\e[2J"); // Clear the Terminal
printf("Start_des_Boards\n");

TaskHandle_t Handle = NULL;
xTaskCreate(start_C1a, "C1a", 128, NULL, 10, &Handle);
//xTaskCreate(start_C1b, "C1b", 128, NULL, 10, &Handle);
//xTaskCreate(start_C2a, "C2a", 128, NULL, 10, &Handle);
//...
/* USER CODE END RTOS_THREADS */

/* USER CODE BEGIN RTOS_EVENTS */
/* add events, ... */
/* USER CODE END RTOS_EVENTS */

/* Start scheduler */
osKernelStart();

/* We should never get here as control is now taken by the scheduler */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
}
```